

ENGG1910 Demystifying AI

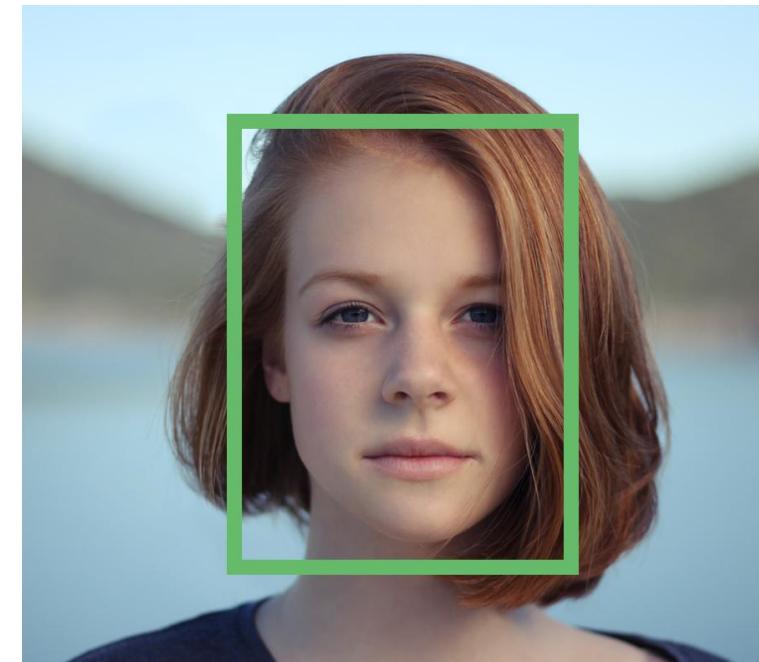
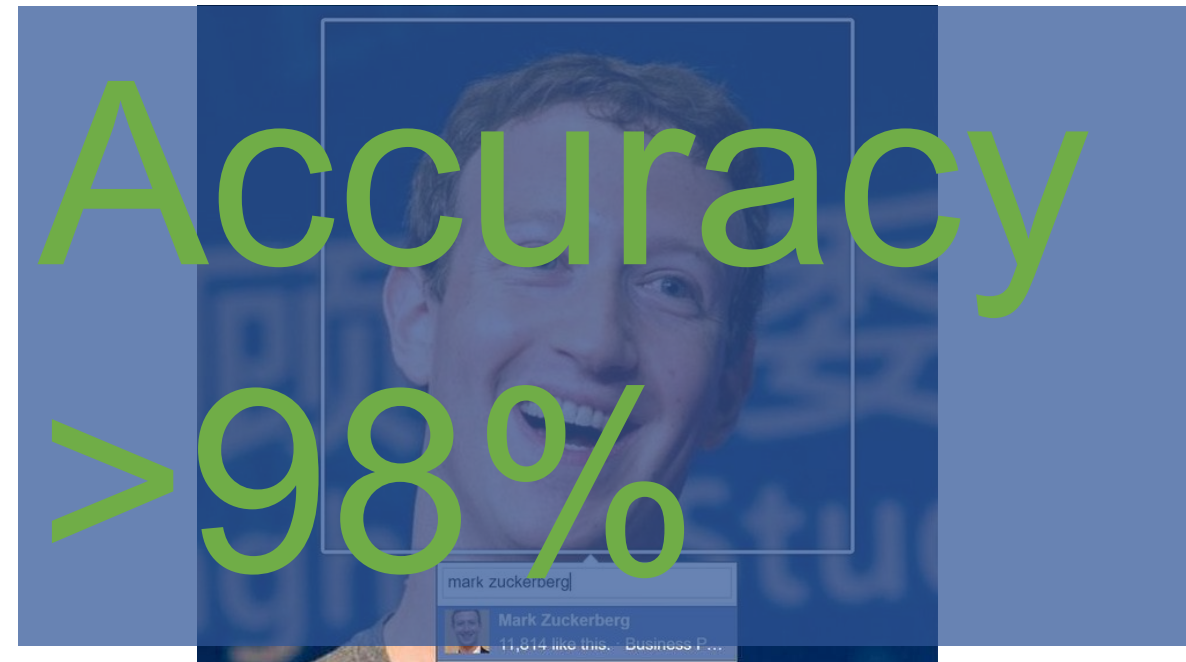
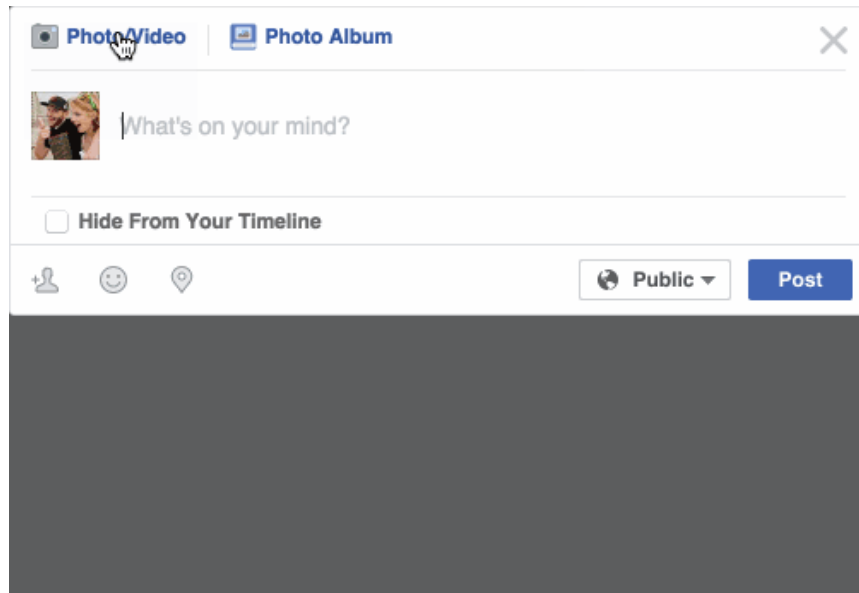
# Lab1 Guidelines

# Outline of Lab1

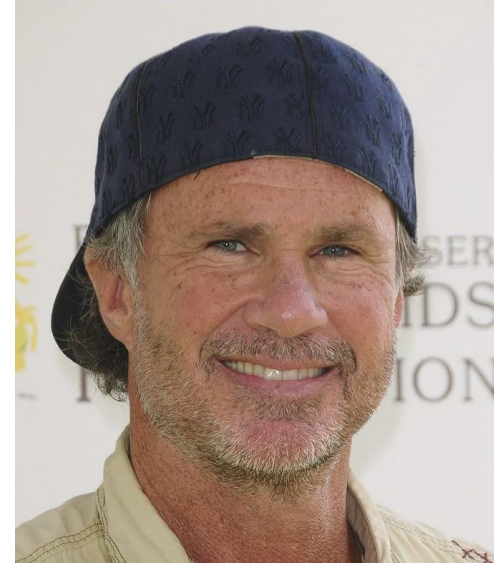
- Familiarize the **procedures** of face recognition with Machine Learning
- Investigate a **demo code** of Face recognition
- Complete **tasks** by studying an open-source project of face recognition

# Motivating examples

- Why it is important?
- How does it work?
- What is the role of machine learning in face recognition?



# Face recognition is not easy



# Series of related problems

- Look at a picture and **find all the faces** in it
- Be able to **identify** even if a face is turned in a weird direction or in bad lighting
- be able to pick out **unique features** of the face that you can use to tell it apart from other people
- Compare the unique features of that face to all the people you already know to determine the person's **name**.

# Face recognition is hard for machines

- Humans are too good at recognizing faces and end up seeing faces in everyday objects:



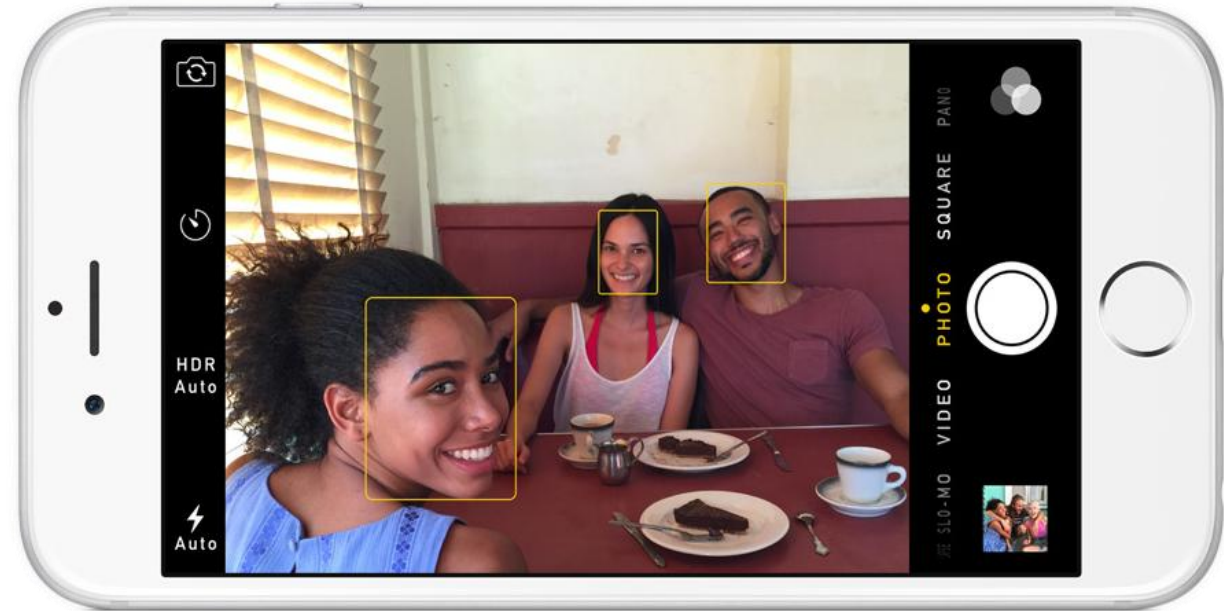
# Steps of face recognition for machines

- **Step 1:** Finding all the Faces
- **Step 2:** Posing and Projecting Faces
- **Step 3:** Encoding Faces
- **Step 4:** Finding the person's name from the encoding

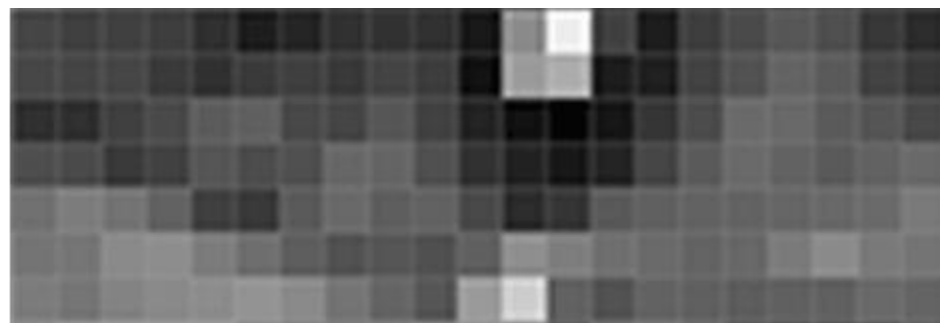
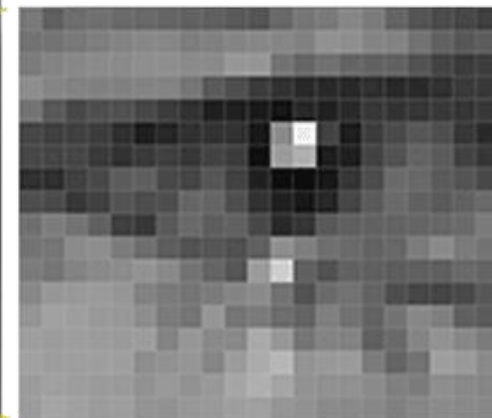
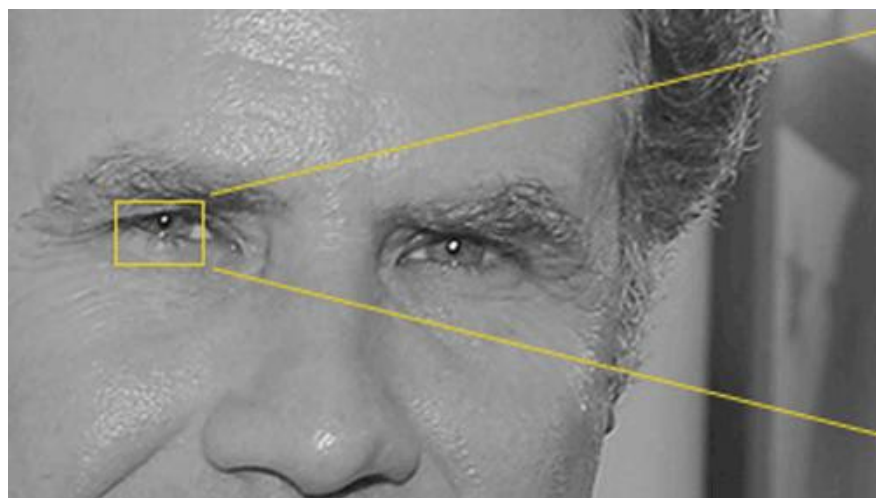


# Step 1: Finding all the Faces

- A basic function of camera in last 15 years
- Face detection went mainstream in the early 2000's when Paul Viola and Michael Jones invented a fast way to detect faces.
- More reliable solution is a method invented in 2005 called **Histogram of Oriented Gradients (HOG)**

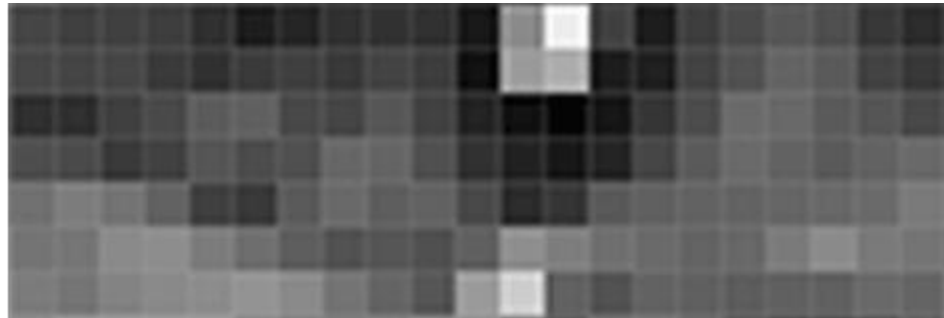


# Histogram of Oriented Gradients (HOG)



# Histogram of Oriented Gradients (HOG)

- Our goal is to figure out how dark the current pixel is compared to the pixels directly surrounding it.
- Then we want to draw an arrow showing in which direction the image is getting darker
- Replace pixels by arrows



# Histogram of Oriented Gradients (HOG)

- If you repeat that process for **every single pixel** in the image, you end up with every pixel being replaced by an arrow.
- These arrows are called **gradients** and they show the flow from light to dark across the entire image



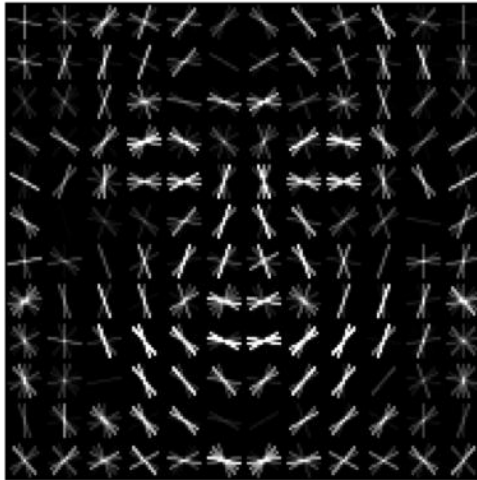
# Representation of pictures in HOG

- Break up the image into small squares of 16x16 pixels each.
- In each square, we'll count up how many gradients point in each major direction

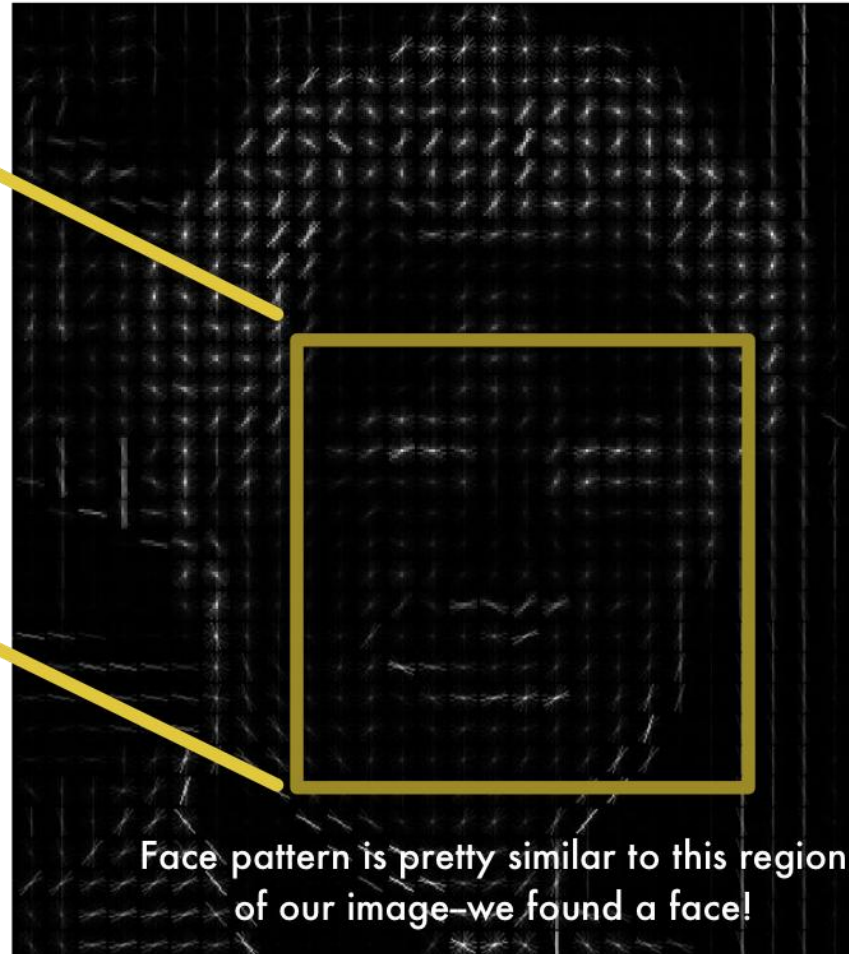


# Find face -> Find HOG face pattern

HOG face pattern generated  
from lots of face images

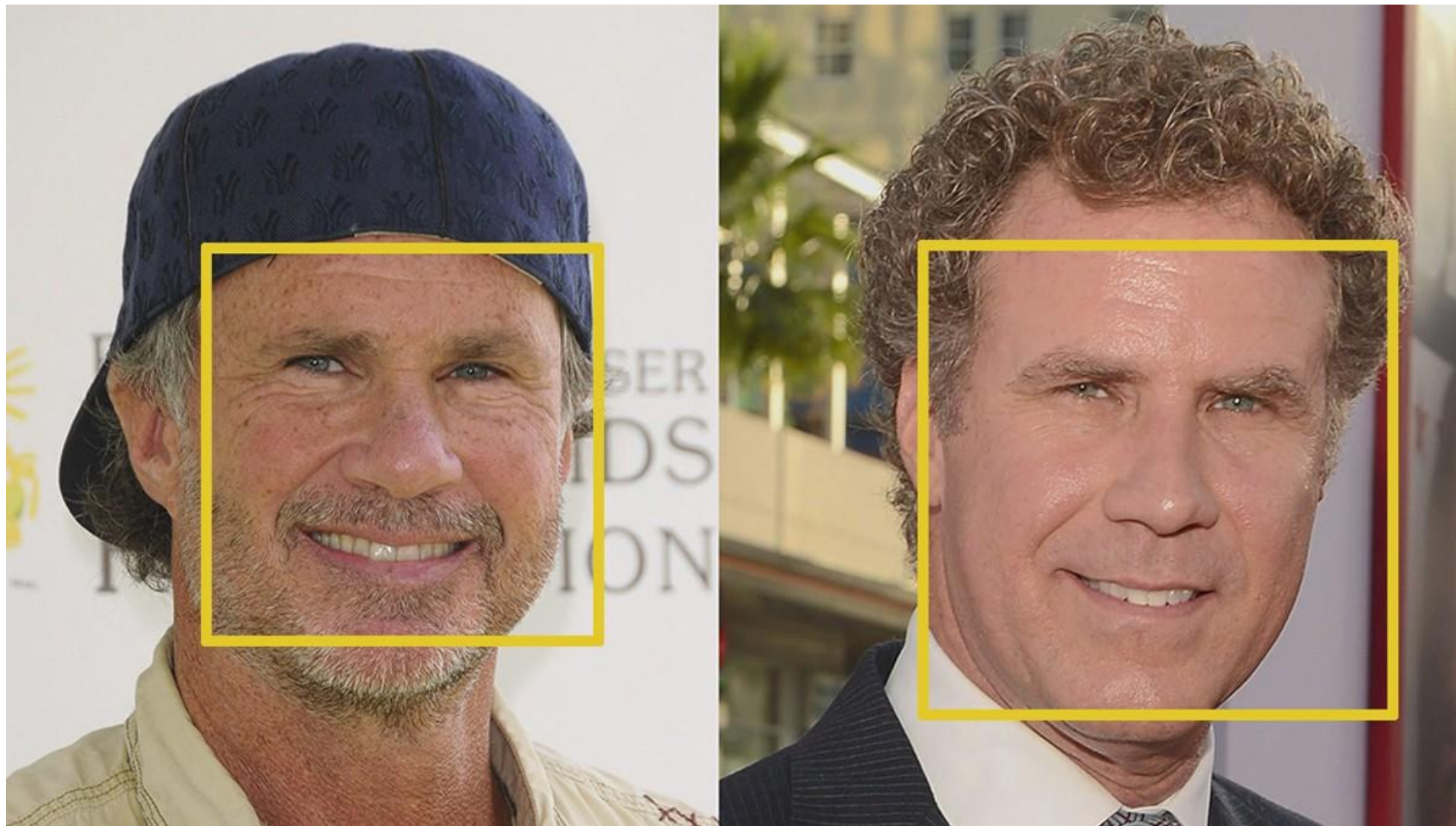


HOG version of our image



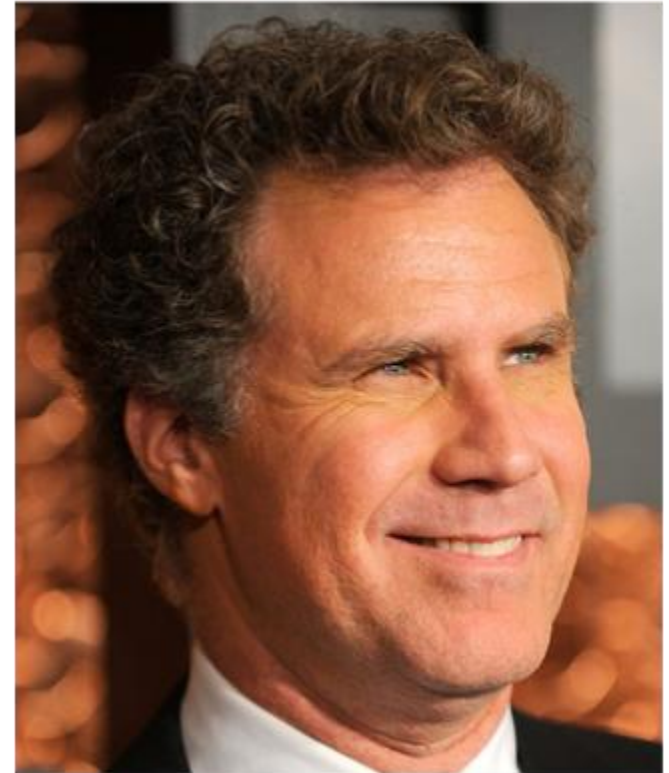
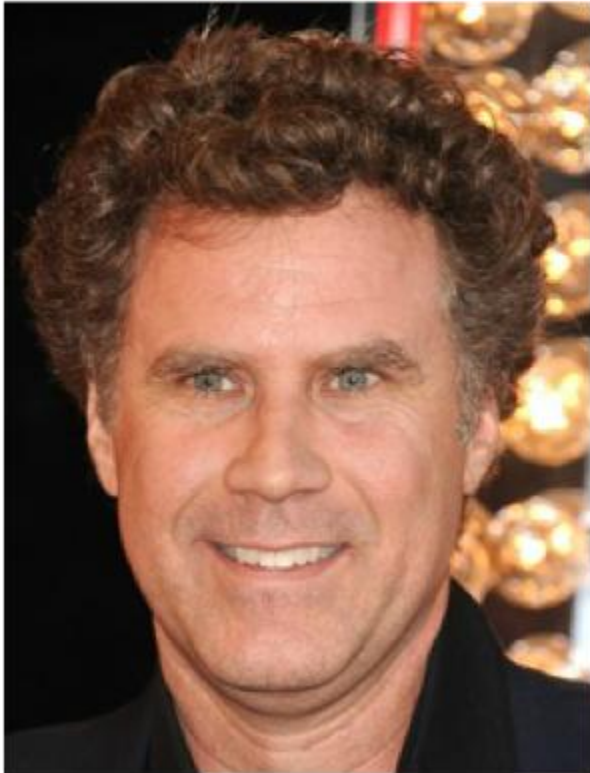
Face pattern is pretty similar to this region  
of our image—we found a face!

# Step 1 completed



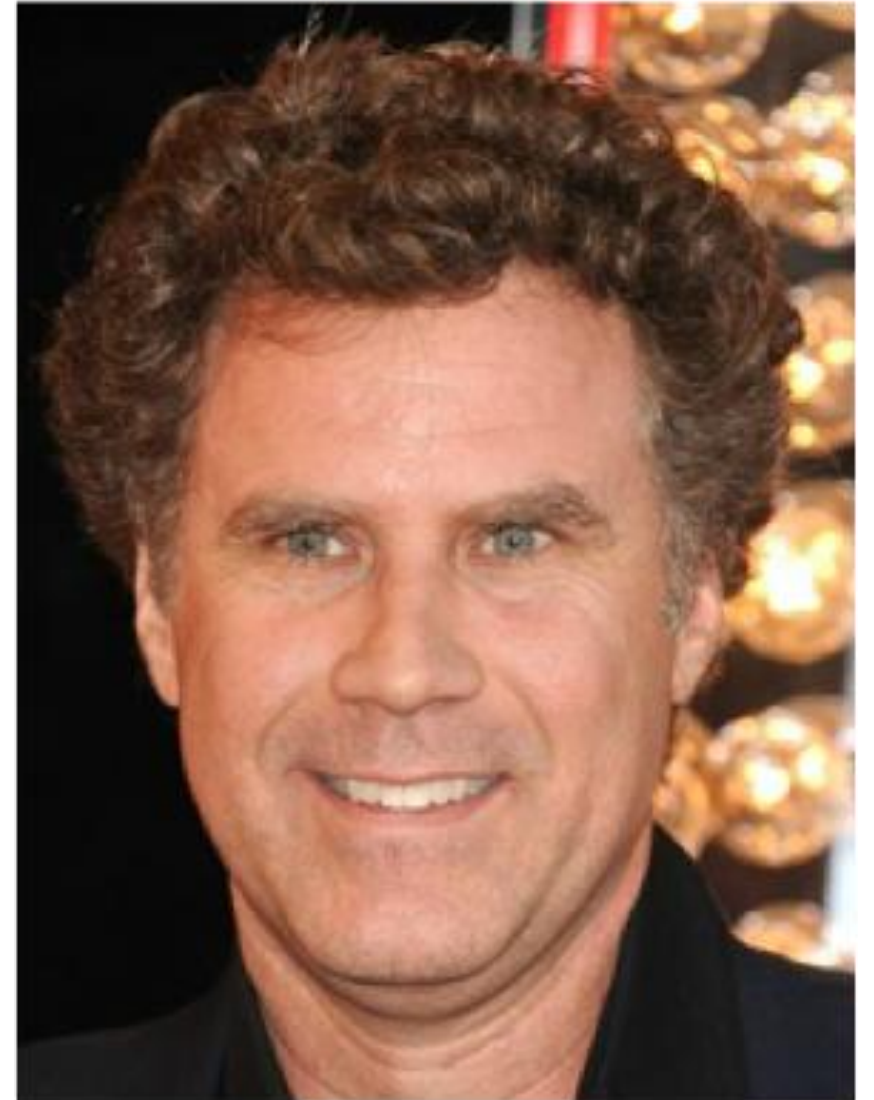
## Step 2: Posing and Projecting Faces

- Now we have to deal with the problem that faces turned different directions look totally different to a computer:

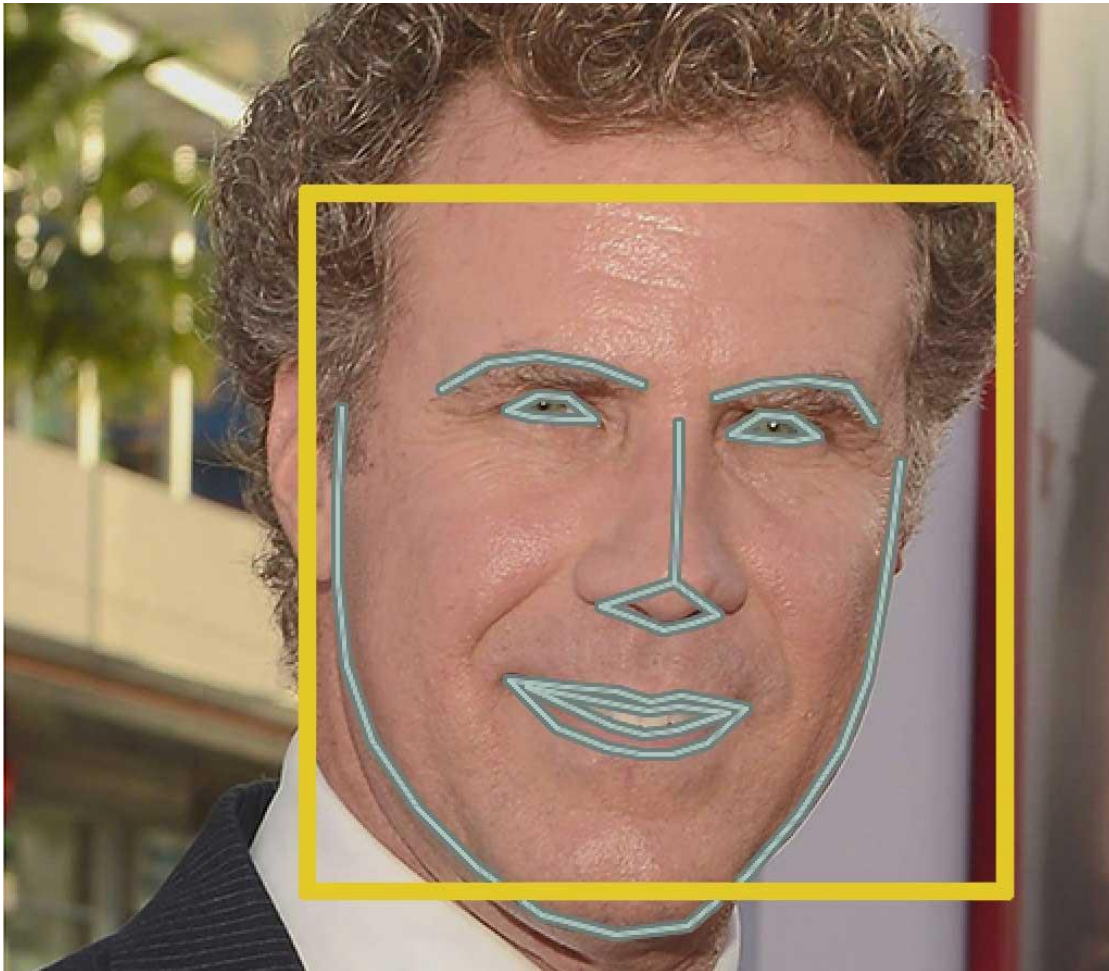


## Step 2: Posing and Projecting Faces

- We are going to use an algorithm called **face landmark estimation**
- The basic idea is we will come up with 68 specific points (called **landmarks**) that exist on every face — the top of the chin, the outside edge of each eye, the inner edge of each eyebrow, etc.
- We will train a machine learning algorithm to be able to find these **68 specific points** on any face



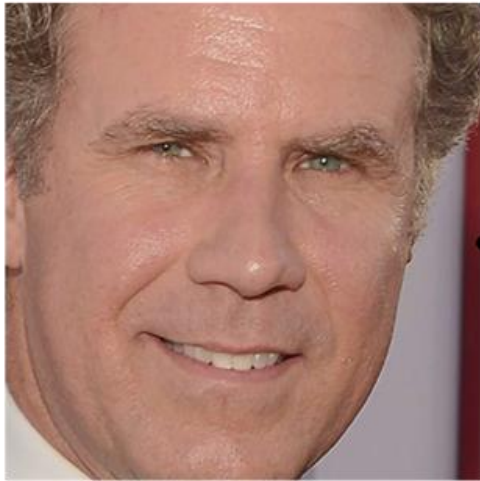
# 68 face landmarks on a test image



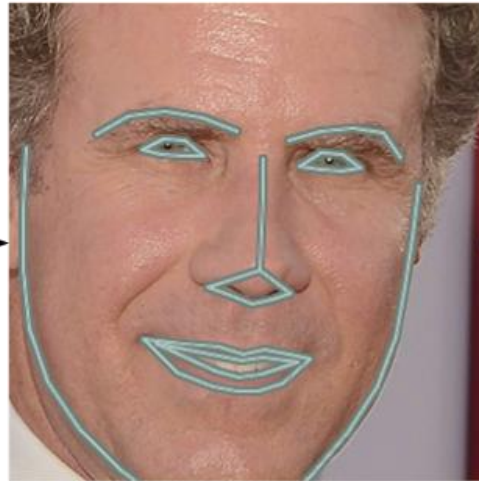
- Basic image transformations like rotation and scale will be tested that preserve parallel lines (**called affine transformations**)
- **Same faces** could be recognized with face landmarks under affine transformation

# 68 face landmarks on a test image

Face area detected in image



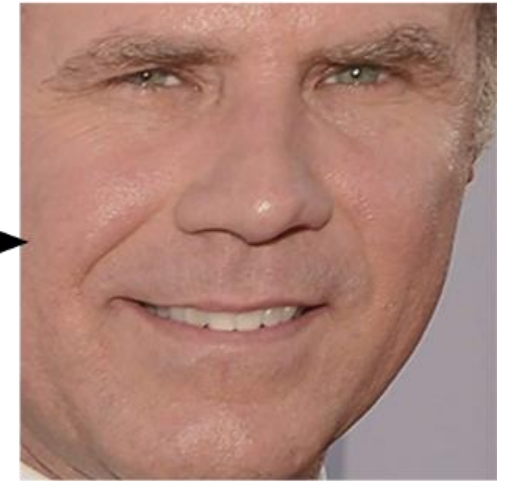
Face landmarks detected



The perfectly centered  
result we want

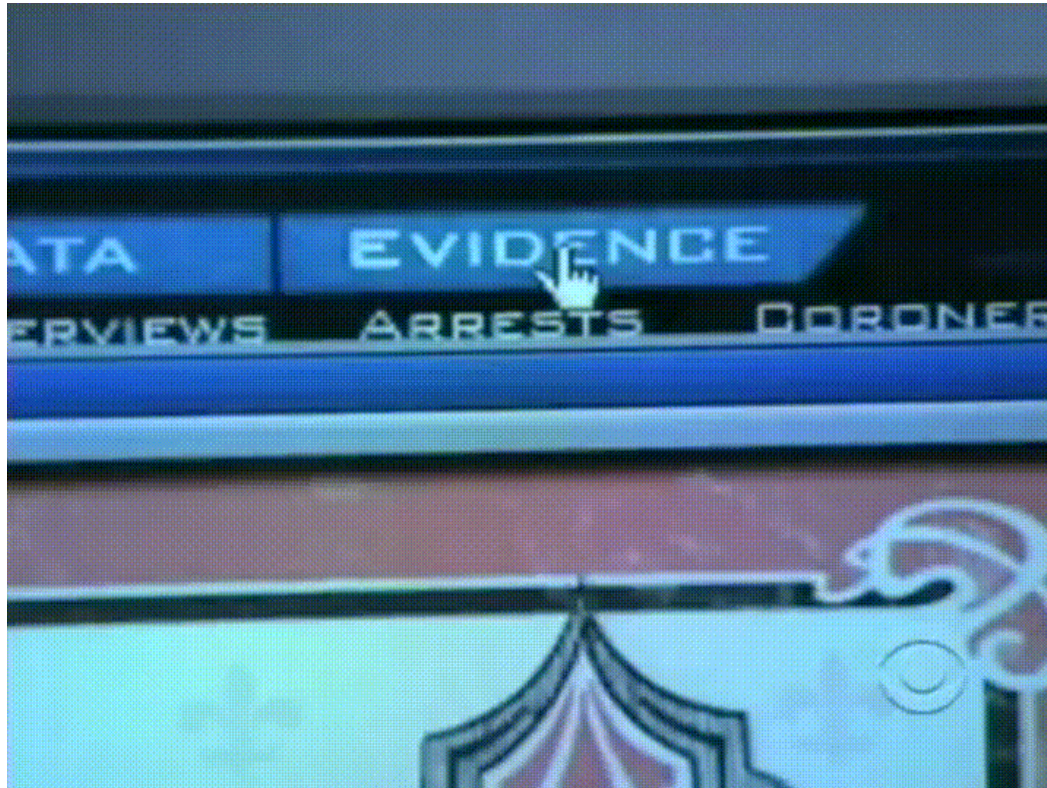


Face transformed to be as close  
as possible to perfectly centered



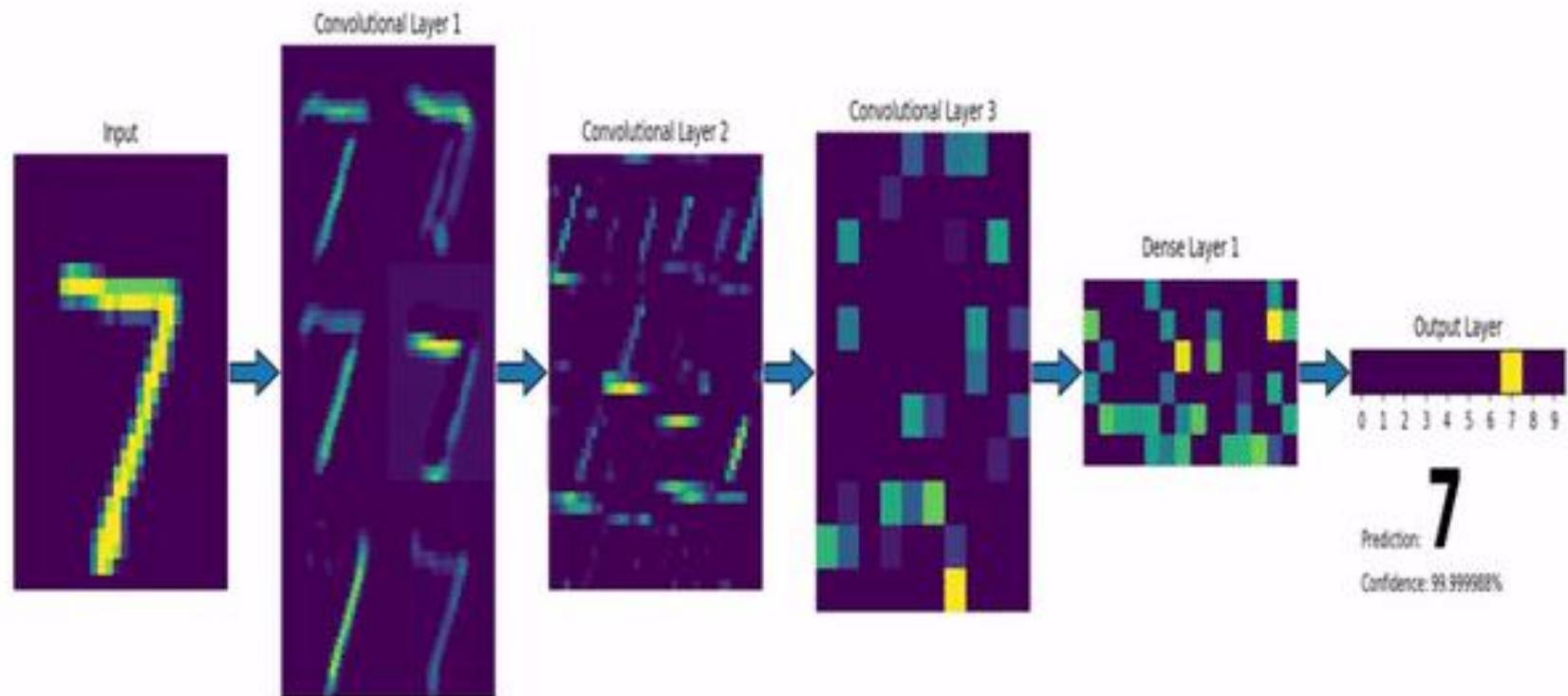
## Step 3: Encoding Faces

Which **measurements** should we collect from each face to build our known face database? Ear size? Nose length? Eye color? Something else?



# Solution: Train a Deep Convolutional Neural Network

- **128 measurements** will be generated for each face
- In machine learning , the 128 measurements are called an **embedding**



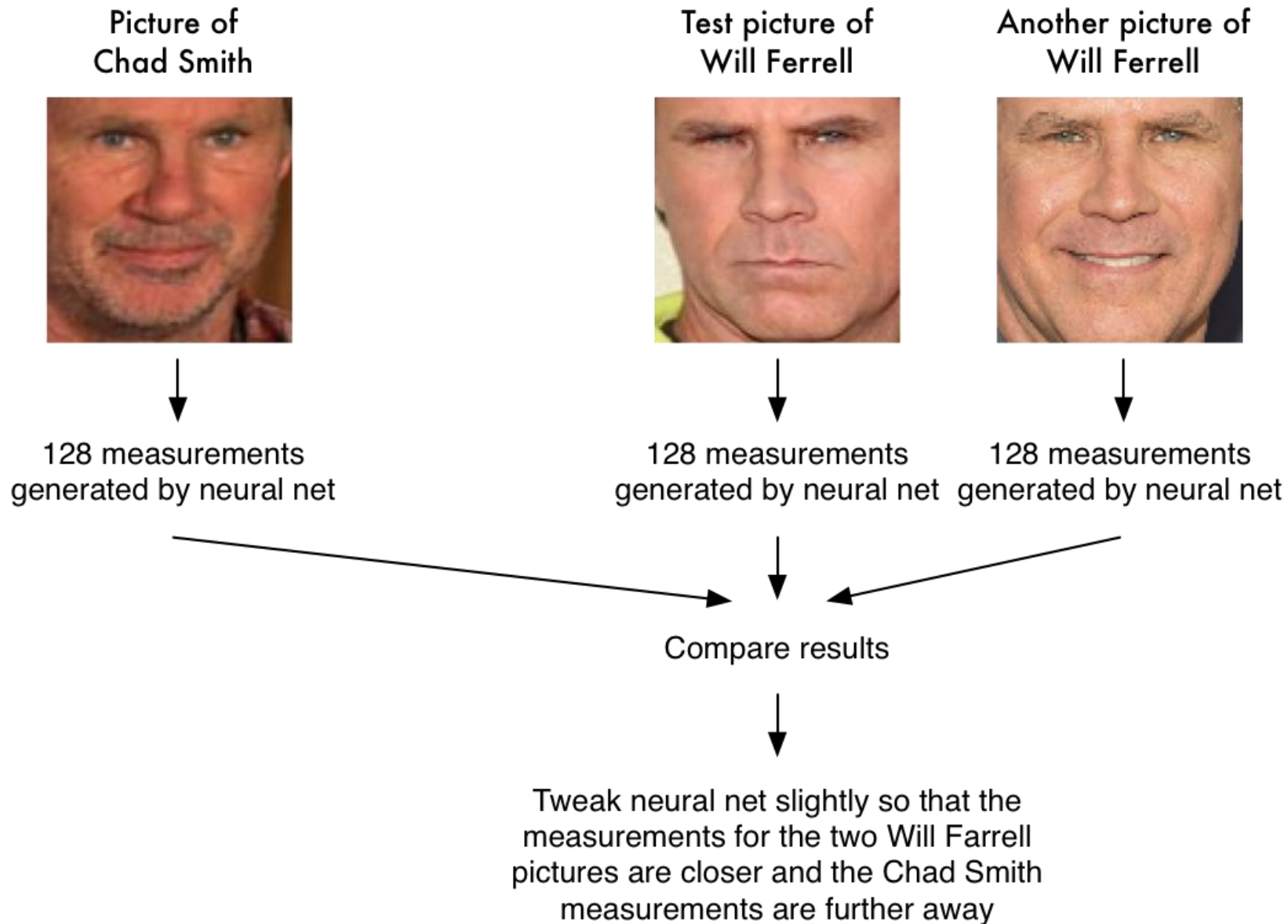
Input Image



## 128 Measurements Generated from Image

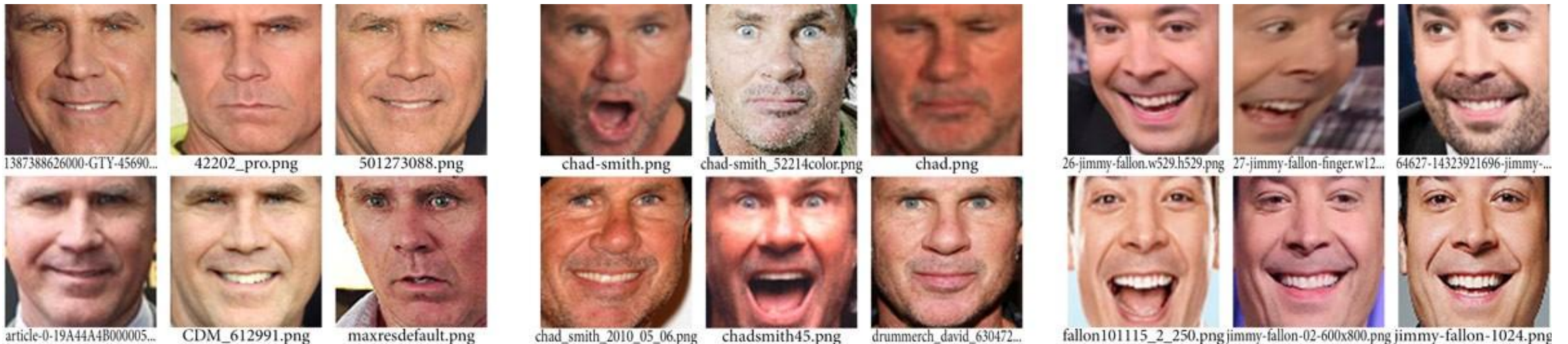
|                     |                     |                    |                      |
|---------------------|---------------------|--------------------|----------------------|
| 0.097496084868908   | 0.045223236083984   | -0.1281466782093   | 0.032084941864014    |
| 0.12529824674129    | 0.060309179127216   | 0.17521631717682   | 0.020976085215807    |
| 0.030809439718723   | -0.01981477253139   | 0.10801389068365   | -0.00052163278451189 |
| 0.036050599068403   | 0.065554238855839   | 0.0731306001544    | -0.1318951100111     |
| -0.097486883401871  | 0.1226262897253     | -0.029626874253154 | -0.0059557510539889  |
| -0.0066401711665094 | 0.036750309169292   | -0.15958009660244  | 0.043374512344599    |
| -0.14131525158882   | 0.14114324748516    | -0.031351584941149 | -0.053343612700701   |
| -0.048540540039539  | -0.061901587992907  | -0.15042643249035  | 0.078198105096817    |
| -0.12567175924778   | -0.10568545013666   | -0.12728653848171  | -0.076289616525173   |
| -0.061418771743774  | -0.074287034571171  | -0.065365232527256 | 0.12369467318058     |
| 0.046741496771574   | 0.0061761881224811  | 0.14746543765068   | 0.056418422609568    |
| -0.12113650143147   | -0.21055991947651   | 0.0041091227903962 | 0.089727647602558    |
| 0.061606746166945   | 0.11345765739679    | 0.021352224051952  | -0.0085843298584223  |
| 0.061989940702915   | 0.19372203946114    | -0.086726233363152 | -0.022388197481632   |
| 0.10904195904732    | 0.084853030741215   | 0.09463594853878   | 0.020696049556136    |
| -0.019414527341723  | 0.0064811296761036  | 0.21180312335491   | -0.050584398210049   |
| 0.15245945751667    | -0.16582328081131   | -0.035577941685915 | -0.072376452386379   |
| -0.12216668576002   | -0.0072777755558491 | -0.036901291459799 | -0.034365277737379   |
| 0.083934605121613   | -0.059730969369411  | -0.070026844739914 | -0.045013956725597   |
| 0.087945111095905   | 0.11478432267904    | -0.089621491730213 | -0.013955107890069   |
| -0.021407851949334  | 0.14841195940971    | 0.078333757817745  | -0.17898085713387    |
| -0.018298890441656  | 0.049525424838066   | 0.13227833807468   | -0.072600327432156   |
| -0.011014151386917  | -0.051016297191381  | -0.14132921397686  | 0.0050511928275228   |
| 0.0093679334968328  | -0.062812767922878  | -0.13407498598099  | -0.014829395338893   |
| 0.058139257133007   | 0.0048638740554452  | -0.039491076022387 | -0.043765489012003   |
| -0.024210374802351  | -0.11443792283535   | 0.071997955441475  | -0.012062266469002   |
| -0.057223934680223  | 0.014683869667351   | 0.05228154733777   | 0.012774495407939    |
| 0.023535015061498   | -0.081752359867096  | -0.031709920614958 | 0.069833360612392    |
| -0.0098039731383324 | 0.037022035568953   | 0.11009479314089   | 0.11638788878918     |
| 0.020220354199409   | 0.12788131833076    | 0.18632389605045   | -0.015336792916059   |
| 0.0040337680839002  | -0.094398014247417  | -0.11768248677254  | 0.10281457751989     |
| 0.051597066223621   | -0.10034311562777   | -0.040977258235216 | -0.082041338086128   |

## A single 'triplet' training step:



# Step 4: Finding the person's name

- This last step is actually the easiest step in the whole process.
- All we have to do is find the person in our database of known people who has the closest measurements to our test image.
- You can do that by using any basic **machine learning classification algorithm**.



# Classification for Tagging



# Review of face recognition

- **Step 1:** Finding all the Faces
- **Step 2:** Posing and Projecting Faces
- **Step 3:** Encoding Faces
- **Step 4:** Finding the person's name from the encoding



# Objectives of Lab1

1. Try to study and run an **open-source project**. (**Objective 1**)
2. In this lab, students are expected to learn the basic procedure of face recognition, including train data and feature vector . (**Objective 2**)
3. Try to make modifications based on open-source projects to achieve specific face recognition . (**Objective 3**)



# Github

1. It is the largest source code host as in 2021.
2. It is commonly used to host open-source projects. As of November 2021, GitHub reports having over 73 million developers and more than 200 million repositories.



# 3 steps of demo code

Step 1: Find faces in pictures

Step 2: Learning from example

Step 3: Showtime



gdg



test



unknown



xyy



xyy\_50



xyy\_90



xyy\_180



xyy\_beard



xyy\_black



xyy\_mask



xyy\_old



xyy\_sex



dongkun1



dongkun2

# Step 1: Find faces in pictures

We start by loading an example picture using Python imaging library. Go ahead and **run the cell below** to see the image.

```
from PIL import Image, ImageDraw
from IPython.display import display

# The program we will be finding faces on the example below
pil_im = Image.open('two_people.jpg')
display(pil_im)
```



# Step 2: Learning from example

Now we show the library two different faces ([Joe Biden](#), [Barack Obama](#)) and generate the encodings for them. Encoding is simply a low dimensional representation of a face that can be easily compared with other faces the library will recognize in the future.

```
import face_recognition
import numpy as np
from PIL import Image, ImageDraw
from IPython.display import display

# This is an example of running face recognition on a single image
# and drawing a box around each person that was identified.

# Load a sample picture and learn how to recognize it.
obama_image = face_recognition.load_image_file("obama.jpg")
obama_face_encoding = face_recognition.face_encodings(obama_image)[0]

# Load a second sample picture and learn how to recognize it.
biden_image = face_recognition.load_image_file("biden.jpg")
biden_face_encoding = face_recognition.face_encodings(biden_image)[0]

# Create arrays of known face encodings and their names
known_face_encodings = [
    obama_face_encoding,
    biden_face_encoding
]
known_face_names = [
    "Barack Obama",
    "Joe Biden"
]

print('Learned encoding for', len(known_face_encodings), 'images.')
```

Learned encoding for 2 images.

# Step 3: Showtime

```
# Load an image with an unknown face
unknown_image = face_recognition.load_image_file("two_people.jpg")

# Find all the faces and face encodings in the unknown image
face_locations = face_recognition.face_locations(unknown_image)
face_encodings = face_recognition.face_encodings(unknown_image, face_locations)

# Convert the image to a PIL-format image so that we can draw on top of it with the Pillow library
# See http://pillow.readthedocs.io/ for more about PIL/Pillow
pil_image = Image.fromarray(unknown_image)
# Create a Pillow ImageDraw Draw instance to draw with
draw = ImageDraw.Draw(pil_image)

# Loop through each face found in the unknown image
for (top, right, bottom, left), face_encoding in zip(face_locations, face_encodings):
    # See if the face is a match for the known face(s)
    matches = face_recognition.compare_faces(known_face_encodings, face_encoding)

    name = "Unknown"

    # Or instead, use the known face with the smallest distance to the new face
    face_distances = face_recognition.face_distance(known_face_encodings, face_encoding)
    best_match_index = np.argmin(face_distances)
    if matches[best_match_index]:
        name = known_face_names[best_match_index]

    # Draw a box around the face using the Pillow module
    draw.rectangle(((left, top), (right, bottom)), outline=(0, 0, 255))

    # Draw a label with a name below the face
    text_width, text_height = draw.textsize(name)
    draw.rectangle(((left, bottom - text_height - 10), (right, bottom)), fill=(0, 0, 255), outline=(0, 0, 255))
    draw.text((left + 6, bottom - text_height - 5), name, fill=(255, 255, 255), font_size=16)

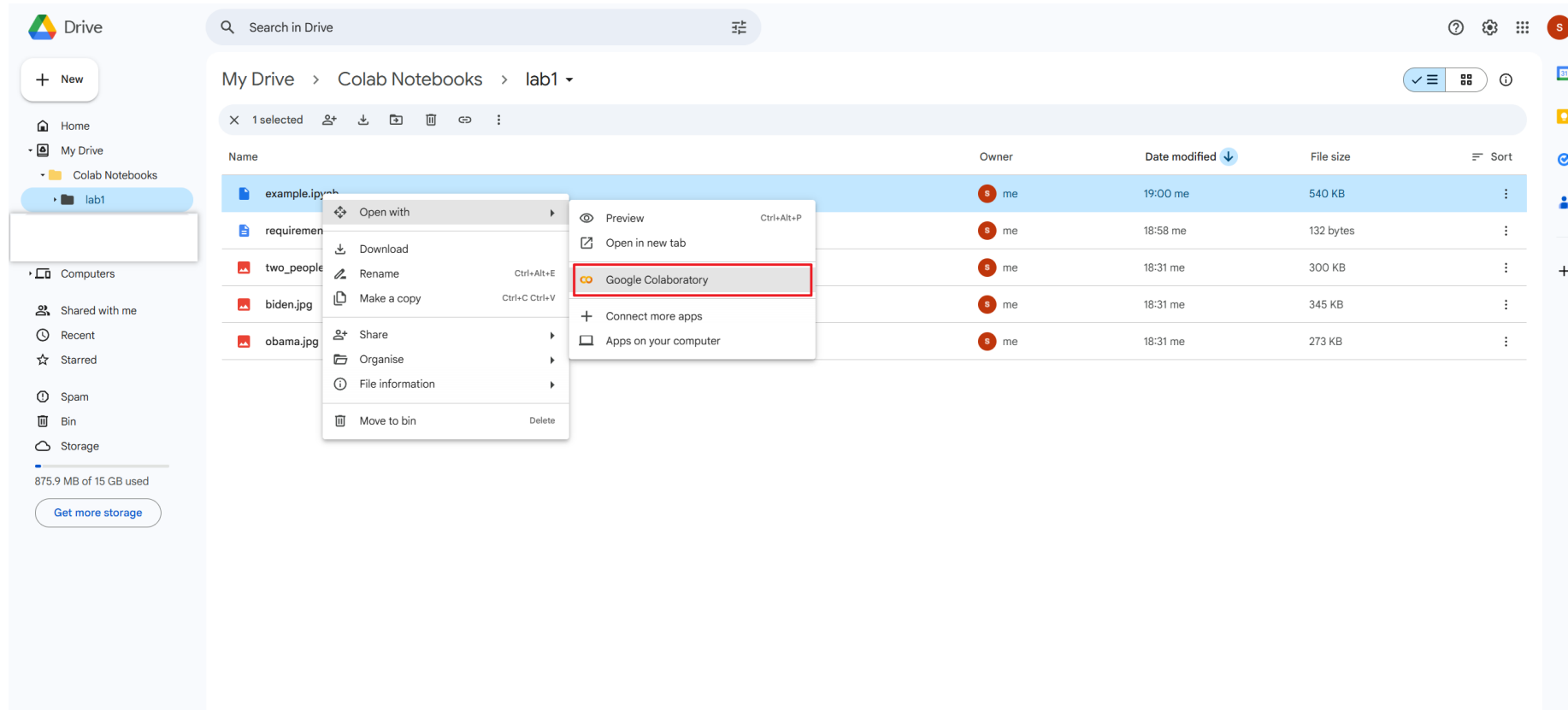
# Remove the drawing library from memory as per the Pillow docs
del draw

# Display the resulting image
display(pil_image)
```



# Task1 -> Read the code and comments

- Download the file Lab1 from Blackboard and unzip the file.
- Upload the folder Lab1 onto your Google drive, in the folder “Colab Notebooks”
- Right-click the “example.ipynb” file again and select “Google Colaboratory” to start the lab.



# Task1 -> Read the code and comments

- Read the code and comments, there are three blocks

## Find faces in pictures

We start by loading an example picture using Python imaging library. Go ahead and **run the cell below** to see the image.

```
from PIL import Image, ImageDraw
from IPython.display import display

# The program we will be finding faces on the example below
pil_im = Image.open('two_people.jpg')
display(pil_im)
```

## Learning from example

Now we show the library two different faces ([Joe Biden](#), [Barack Obama](#)) and generate the encodings for them. Encoding is simply a low dimensional representation of a face that can be easily compared with other faces the library will recognize in the future.

```
1 import face_recognition
2 import numpy as np
3 from PIL import Image, ImageDraw
4 from IPython.display import display
5
6 # This is an example of running face recognition on a single image
7 # and drawing a box around each person that was identified.
8
9 # Load a sample picture and learn how to recognize it.
10 obama_image = face_recognition.load_image_file("obama.jpg")
11 obama_face_encoding = face_recognition.face_encodings(obama_image)[0]
12
13 # Load a second sample picture and learn how to recognize it.
14 biden_image = face_recognition.load_image_file("biden.jpg")
15 biden_face_encoding = face_recognition.face_encodings(biden_image)[0]
16
17 # Create arrays of known face encodings and their names
18 known_face_encodings = [
19     obama_face_encoding,
20     biden_face_encoding
21 ]
22 known_face_names = [
23     "Barack Obama",
24     "Joe Biden"
25 ]
26 print('Learned encoding for', len(known_face_encodings), 'images.')
```

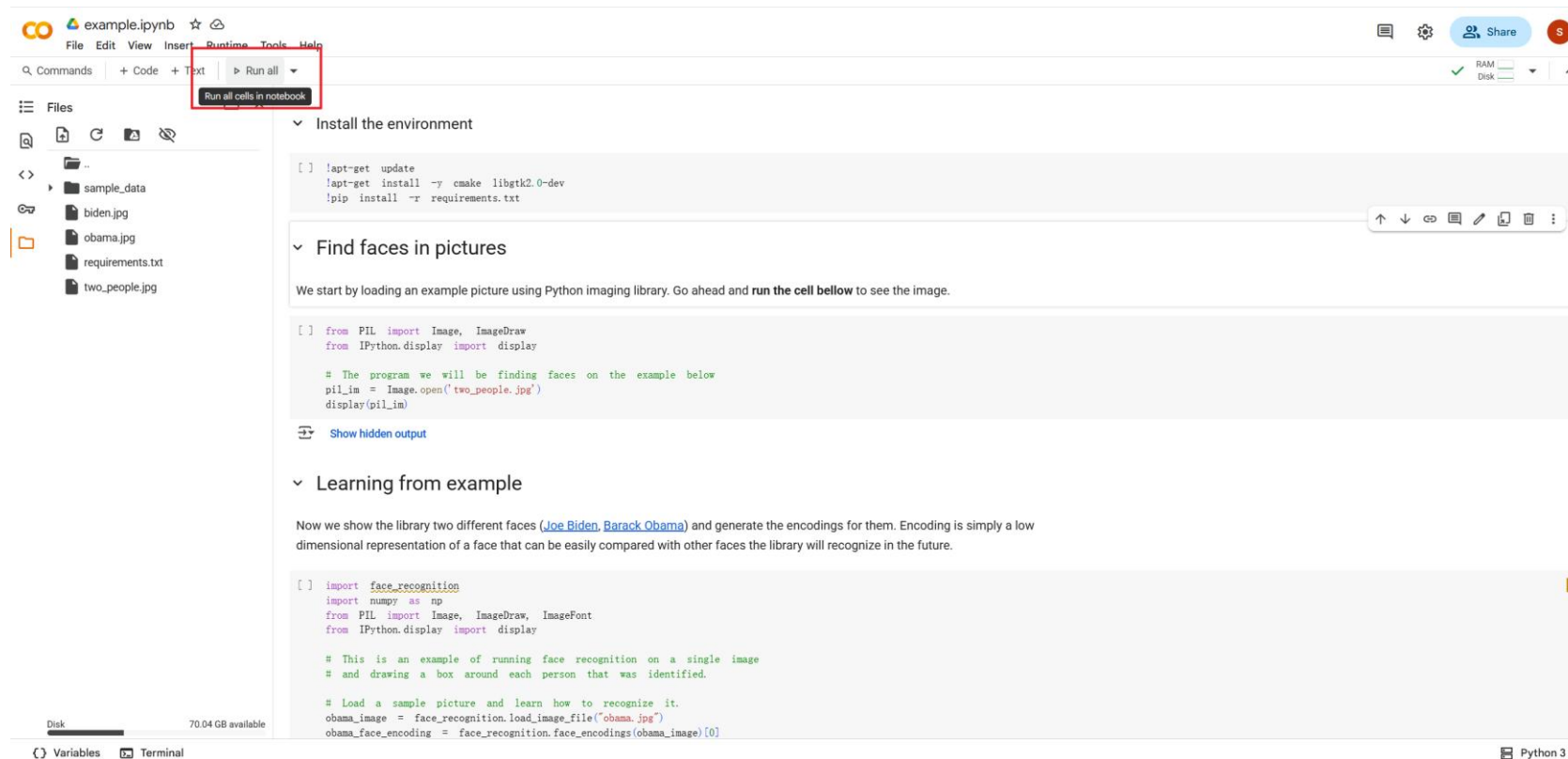
## Showtime

Finally, we load the image we looked at in the first cell, find the faces in the image and compare them with the encodings the library generated in the previous step. We can see that library now correctly recognizes Barack and Joe in the input.

```
# Load an image with an unknown face
unknown_image = face_recognition.load_image_file("two_people.jpg")
```

# Task2 -> Run the demo code

- Just click “Run all” and wait for several seconds
- Under the “find faces in pictures”,
  - show the picture with several people who we would like to recognize
- Under the “show time”, faces are boxed out and labeled with his name



# Task2 -> Run the demo code

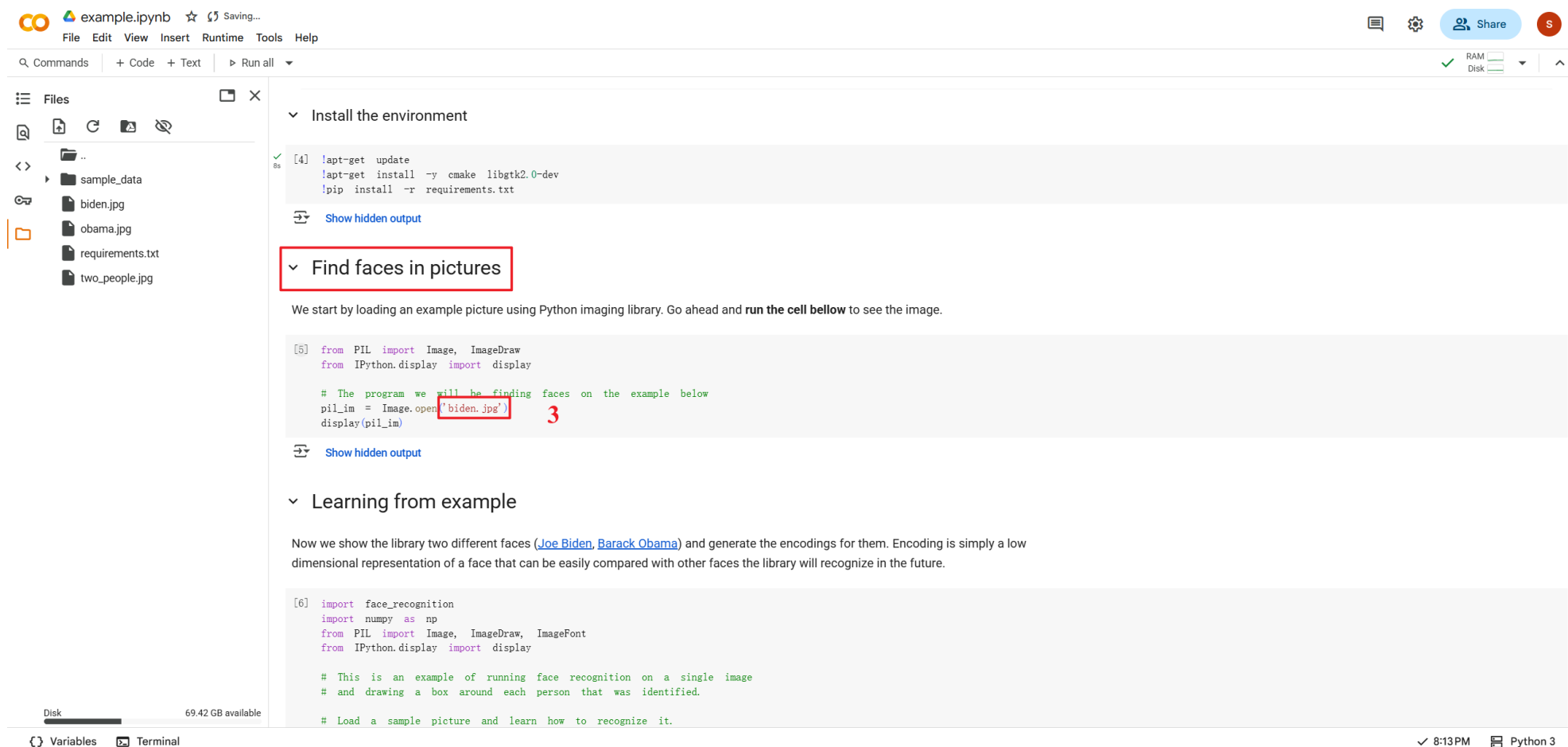
- Select the image name 'biden.jpg' to replace the original image name 'two\_people.jpg'.
- Run the demo again.

The screenshot shows a Jupyter Notebook interface with the following components:

- Files Panel (Left):** Displays a file tree with a folder named 'sample\_data'. Inside this folder, the file 'biden.jpg' is highlighted with a red box and a red number '1' next to it. Other files listed are 'obama.jpg', 'requirements.txt', and 'two\_people.jpg'.
- Code Cells:**
  - Cell [4]:** Titled 'Install the environment', it contains commands to update apt, install cmake and libgtk2.0-dev, and install requirements.txt. It has a 'Show hidden output' link.
  - Cell [5]:** Titled 'Find faces in pictures', it contains Python code to load an image using PIL. The line `pil_im = Image.open('two_people.jpg')` has 'two\_people.jpg' highlighted with a red box and a red number '2' next to it. Below the code is a 'Show hidden output' link.
  - Cell [6]:** Titled 'Learning from example', it contains code to import face\_recognition, numpy, PIL, and IPython.display. It includes comments about running face recognition on a single image and drawing a box around identified persons.
- Bottom Bar:** Shows '69.42 GB available' on the left, 'Variables' and 'Terminal' tabs in the middle, and '8:13 PM Python 3' on the right.

# Task2 -> Run the demo code

- Select the image name 'biden.jpg' to replace the original image name 'two\_people.jpg'.
- Run the demo again.



The screenshot shows a JupyterLab interface with a notebook titled 'example.ipynb'. The left sidebar displays a file explorer with the following files: 'sample\_data', 'biden.jpg', 'obama.jpg', 'requirements.txt', and 'two\_people.jpg'. The main area contains two code cells.

**Cell 1: Install the environment**

```
[4] !apt-get update
    !apt-get install -y cmake libgtk2.0-dev
    !pip install -r requirements.txt
```

**Cell 2: Find faces in pictures**

We start by loading an example picture using Python imaging library. Go ahead and **run the cell below** to see the image.

```
[5] from PIL import Image, ImageDraw
    from IPython.display import display

    # The program we will be finding faces on the example below
    pil_im = Image.open('biden.jpg')
    display(pil_im)
```

**Cell 3: Learning from example**

Now we show the library two different faces ([Joe Biden](#), [Barack Obama](#)) and generate the encodings for them. Encoding is simply a low dimensional representation of a face that can be easily compared with other faces the library will recognize in the future.

```
[6] import face_recognition
    import numpy as np
    from PIL import Image, ImageDraw, ImageFont
    from IPython.display import display

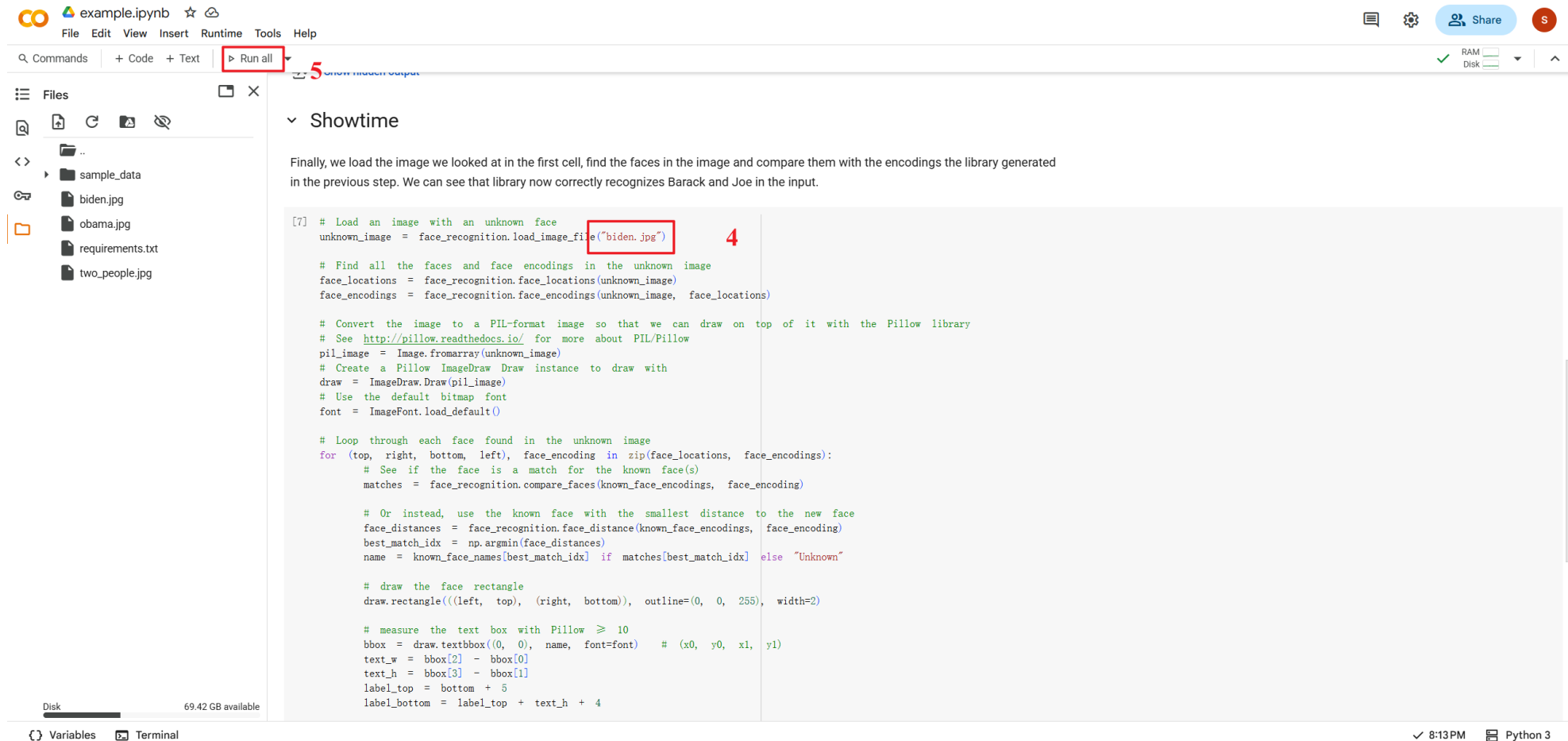
    # This is an example of running face recognition on a single image
    # and drawing a box around each person that was identified.

    # Load a sample picture and learn how to recognize it.
```

The bottom status bar shows 'Disk 69.42 GB available', 'Variables', 'Terminal', '8:13 PM', and 'Python 3'.

# Task2 -> Run the demo code

- Select the image name 'biden.jpg' to replace the original image name 'two\_people.jpg'.
- Run the demo again.



The screenshot shows a Jupyter Notebook interface with a file explorer on the left and a code editor in the center. The file explorer lists files: sample\_data, biden.jpg, obama.jpg, requirements.txt, and two\_people.jpg. The code editor contains a Python script for face recognition. A red box highlights the filename 'biden.jpg' in the code, and a red number '4' is next to it. The script includes comments and code for loading an image, finding faces, comparing face encodings, and drawing a rectangle around the face.

```
[7] # Load an image with an unknown face
unknown_image = face_recognition.load_image_file("biden.jpg")

# Find all the faces and face encodings in the unknown image
face_locations = face_recognition.face_locations(unknown_image)
face_encodings = face_recognition.face_encodings(unknown_image, face_locations)

# Convert the image to a PIL-format image so that we can draw on top of it with the Pillow library
# See http://pillow.readthedocs.io/ for more about PIL/Pillow
pil_image = Image.fromarray(unknown_image)
# Create a Pillow ImageDraw Draw instance to draw with
draw = ImageDraw.Draw(pil_image)
# Use the default bitmap font
font = ImageFont.load_default()

# Loop through each face found in the unknown image
for (top, right, bottom, left), face_encoding in zip(face_locations, face_encodings):
    # See if the face is a match for the known face(s)
    matches = face_recognition.compare_faces(known_face_encodings, face_encoding)

    # Or instead, use the known face with the smallest distance to the new face
    face_distances = face_recognition.face_distance(known_face_encodings, face_encoding)
    best_match_idx = np.argmin(face_distances)
    name = known_face_names[best_match_idx] if matches[best_match_idx] else "Unknown"

    # draw the face rectangle
    draw.rectangle((left, top), (right, bottom), outline=(0, 0, 255), width=2)

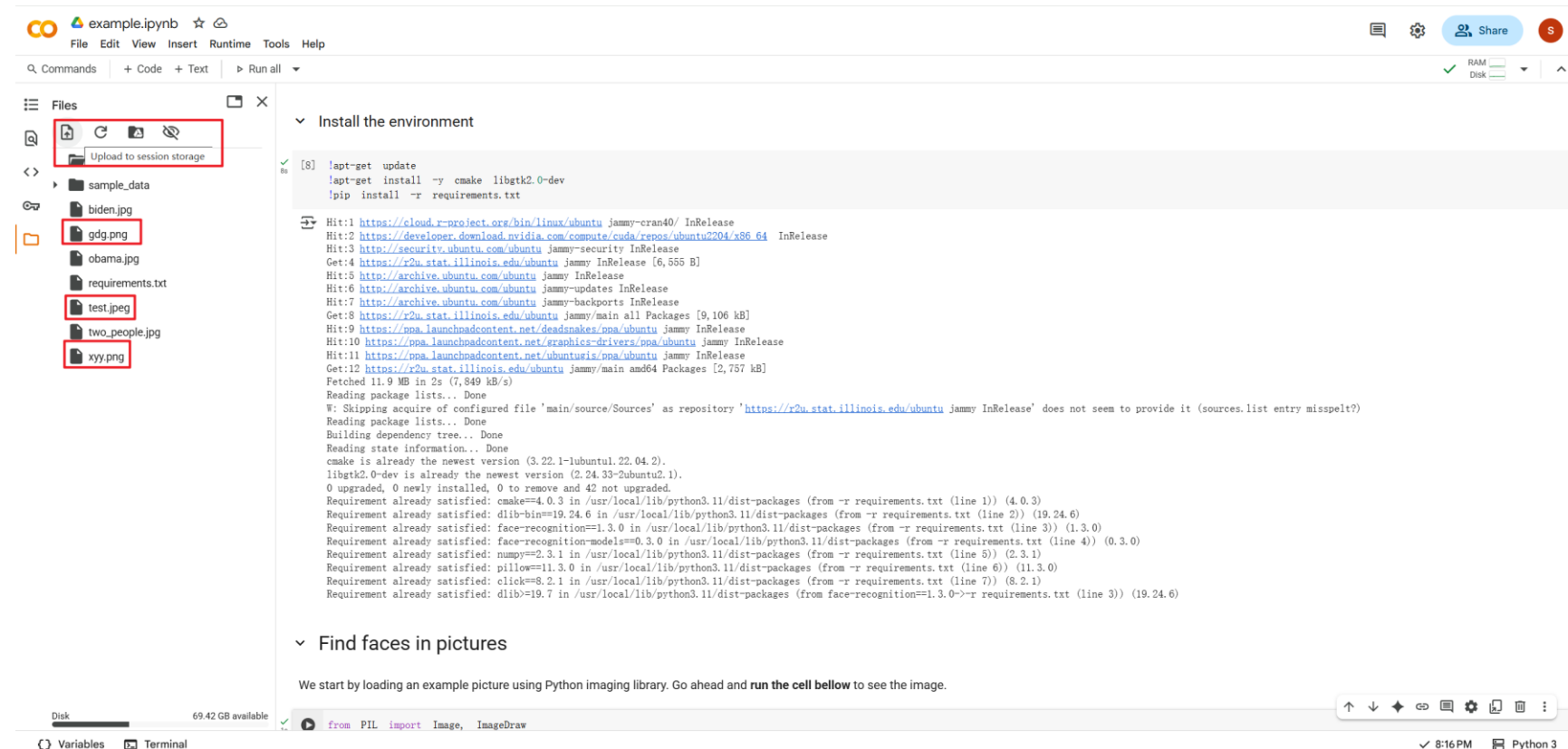
    # measure the text box with Pillow >= 10
    bbox = draw.textbbox((0, 0), name, font=font) # (x0, y0, x1, y1)
    text_w = bbox[2] - bbox[0]
    text_h = bbox[3] - bbox[1]
    label_top = bottom + 5
    label_bottom = label_top + text_h + 4
```

8:13 PM Python 3

# Task3 -> Add your image and extract the latent code

## Add your images

- Example
  - Here I add 'gdg.png' and 'xyy.png', you can add the images you like.



# Task3 -> Add your image and extract the latent code

## Learning from example

Now we show the library two different faces ([Joe Biden](#), [Barack Obama](#)) and generate the encodings for them. Encoding is simply a low dimensional representation of a face that can be easily compared with other faces the library will recognize in the future.

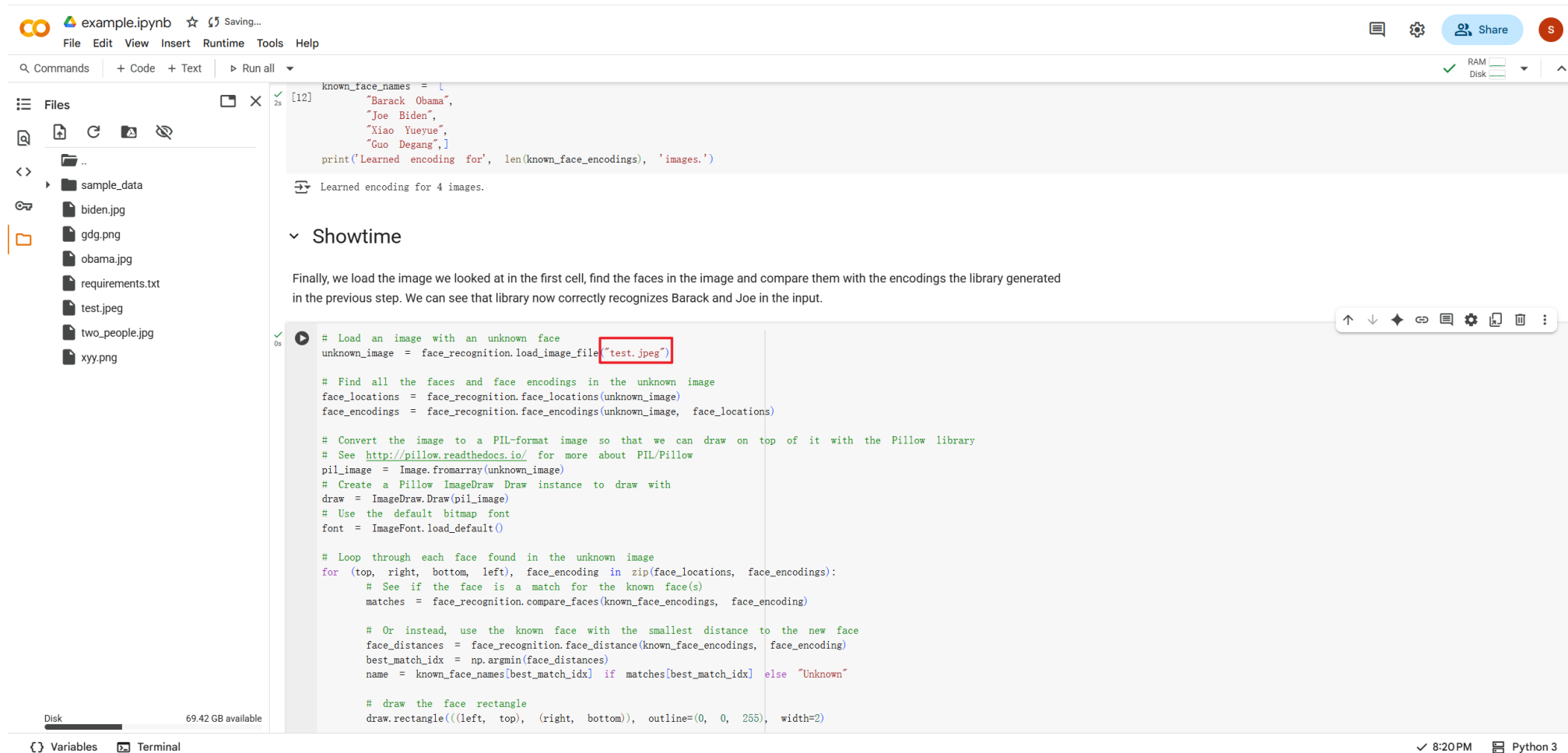
```
1 import face_recognition
2 import numpy as np
3 from PIL import Image, ImageDraw
4 from IPython.display import display
5
6 # This is an example of running face recognition on a single image
7 # and drawing a box around each person that was identified.
8
9 # Load a sample picture and learn how to recognize it.
10 obama_image = face_recognition.load_image_file("obama.jpg")
11 obama_face_encoding = face_recognition.face_encodings(obama_image)[0]
12
13 # Load a second sample picture and learn how to recognize it.
14 biden_image = face_recognition.load_image_file("biden.jpg")
15 biden_face_encoding = face_recognition.face_encodings(biden_image)[0]
16
17 # Load a sample picture and learn how to recognize it.
18 xyy_image = face_recognition.load_image_file("xyy.png")
19 xyy_face_encoding = face_recognition.face_encodings(xyy_image)[0]
20
21 # Load a second sample picture and learn how to recognize it.
22 gdg_image = face_recognition.load_image_file("gdg.png")
23 gdg_face_encoding = face_recognition.face_encodings(gdg_image)[0]
24
25 # Create arrays of known face encodings and their names
26 known_face_encodings = [
27     obama_face_encoding,
28     biden_face_encoding,
29     xyy_face_encoding,
30     gdg_face_encoding,
31 ]
32 known_face_names = [
33     "Barack Obama",
34     "Joe Biden",
35     "Xiao Yueyue",
36     "Guo Degang",
37 ]
38 print('Learned encoding for', len(known_face_encodings), 'images.')
```

Learned encoding for 4 images.

Run  
Last n  
Add o  
Share  
Delete  
Block

# Task3 -> Test on your image

- Add the test image to the dataset, e.g., “test.jpg”.
- Click “run” to get the result and this is the test time.



The screenshot displays a Jupyter Notebook environment with the following components:

- Files Panel (Left):** Shows a directory structure with files including `sample_data`, `biden.jpg`, `gdg.png`, `obama.jpg`, `requirements.txt`, `test.jpeg` (highlighted), `two_people.jpg`, and `xyy.png`.
- Code Cell (Top):** Contains a list of known face names and a print statement to show the number of learned encodings.

```
known_face_names = [
    "Barack Obama",
    "Joe Biden",
    "Xiao Yueyue",
    "Guo Degang",
]
print('Learned encoding for', len(known_face_encodings), 'images.')
```

The output shows: `Learned encoding for 4 images.`
- Section Header:** A heading `Showtime` is present.
- Text:** A paragraph explaining the next step: "Finally, we load the image we looked at in the first cell, find the faces in the image and compare them with the encodings the library generated in the previous step. We can see that library now correctly recognizes Barack and Joe in the input."
- Code Cell (Bottom):** Contains the main logic for loading, processing, and recognizing faces in `test.jpeg`. The filename `"test.jpeg"` is highlighted with a red box.

```
# Load an image with an unknown face
unknown_image = face_recognition.load_image_file("test.jpeg")

# Find all the faces and face encodings in the unknown image
face_locations = face_recognition.face_locations(unknown_image)
face_encodings = face_recognition.face_encodings(unknown_image, face_locations)

# Convert the image to a PIL-format image so that we can draw on top of it with the Pillow library
# See http://pillow.readthedocs.io/ for more about PIL/Pillow
pil_image = Image.fromarray(unknown_image)
# Create a Pillow ImageDraw Draw instance to draw with
draw = ImageDraw.Draw(pil_image)
# Use the default bitmap font
font = ImageFont.load_default()

# Loop through each face found in the unknown image
for (top, right, bottom, left), face_encoding in zip(face_locations, face_encodings):
    # See if the face is a match for the known face(s)
    matches = face_recognition.compare_faces(known_face_encodings, face_encoding)

    # Or instead, use the known face with the smallest distance to the new face
    face_distances = face_recognition.face_distance(known_face_encodings, face_encoding)
    best_match_idx = np.argmin(face_distances)
    name = known_face_names[best_match_idx] if matches[best_match_idx] else "Unknown"

# draw the face rectangle
draw.rectangle(((left, top), (right, bottom)), outline=(0, 0, 255), width=2)
```
- Footer:** Shows system status (8:20 PM), Python version (Python 3), and available disk space (69.42 GB available).

# Task3 -> Test on your image

- Add the test image to the dataset, e.g., “test.jpg”.
- Click “run” to get the result and this is the test time.



# Task4 -> Image after transformation

We prepared some processed images including color changes, angle changes, face changes, etc. We can observe the impact on the face recognition system.

- Rotate  
different angles: 50°, 90°, 180° (xyy\_50.png, xyy\_90.png, xyy\_180.png)
- black and white  
xyy\_black.png
- Older  
xyy\_old.png
- Sex change  
xyy\_sex.png
- face change  
xyy\_beard.png
- Mask  
xyy\_mask.png

Crop the eight results under res/task4/res\_xyy\_\*.png, where \* is 50, 80, 180, .... , mask

# Task5 -> Test on an image who not in the dataset

- I prepared two additional images for the test, the people in the images are not in our database.

“unknown.jpg and dongkun1.jpg “



# Task5 -> Test on an image who not in the dataset

Files

sample\_data

biden.jpg

dongkun1.jpg

dongkun2.jpg

example.ipynb

gdg.png

obama.jpg

requirements.txt

test.jpeg

two\_people.jpg

unknown.jpg

xyy.png

xyy\_180.png

xyy\_50.png

xyy\_90.png

xyy\_beard.png

xyy\_black.png

xyy\_mask.png

xyy\_old.png

xyy\_sex.png

0s

2. Click "run" button

```
# Load an image with an unknown face
unknown_image = face_recognition.load_image_file("unknown.jpg")

# Find all the faces and face encodings in the unknown image
face_locations = face_recognition.face_locations(unknown_image)
face_encodings = face_recognition.face_encodings(unknown_image, face_locations)

# Convert the image to a PIL-format image so that we can draw on top of it with the Pillow library
# See http://pillow.readthedocs.io/ for more about PIL/Pillow
pil_image = Image.fromarray(unknown_image)
# Create a Pillow ImageDraw Draw instance to draw with
draw = ImageDraw.Draw(pil_image)
# Use the default bitmap font
font = ImageFont.load_default()

# Loop through each face found in the unknown image
for (top, right, bottom, left), face_encoding in zip(face_locations, face_encodings):
    # See if the face is a match for the known face(s)
    matches = face_recognition.compare_faces(known_face_encodings, face_encoding)

    # Or instead, use the known face with the smallest distance to the new face
    face_distances = face_recognition.face_distance(known_face_encodings, face_encoding)
    best_match_idx = np.argmin(face_distances)
    name = known_face_names[best_match_idx] if matches[best_match_idx] else "Unknown"

    # draw the face rectangle
    draw.rectangle(((left, top), (right, bottom)), outline=(0, 0, 255), width=2)

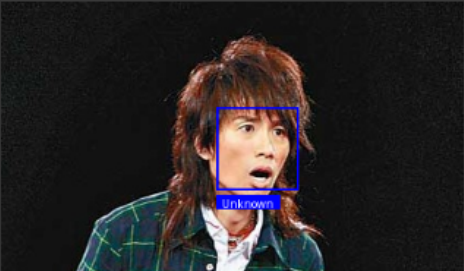
    # measure the text box with Pillow >= 10
    bbox = draw.textbbox((0, 0), name, font=font) # (x0, y0, x1, y1)
    text_w = bbox[2] - bbox[0]
    text_h = bbox[3] - bbox[1]
    label_top = bottom + 5
    label_bottom = label_top + text_h + 4

    # draw the background rectangle and add the label
    draw.rectangle(((left, label_top), (left + text_w + 10, label_bottom)),
        fill=(0, 0, 255), outline=(0, 0, 255))
    draw.text((left + 5, label_top + 2), name,
        fill=(255, 255, 255), font=font)

del draw # release Pillow drawing object
display(pil_image)
```

1. Choose one image, who is not in the database. Here is an example "unknown.jpg"

3. Screenshot your codes and recognized result. Paste them in your report



Disk 69.42 GB available

Task5 -> Test on an image who not in the dataset



dongkun1.jpg

# Task5 -> Test on an image who not in the dataset

Add Dongkun's image in our dataset. Test again.

```
# Load gdg sample picture and learn how to recognize it.
gdg_image = face_recognition.load_image_file("gdg.png")
gdg_face_encoding = face_recognition.face_encodings(gdg_image)[0]

# Load gdg sample picture and learn how to recognize it.
dongkun_image = face_recognition.load_image_file("dongkun2.jpg")
dongkun_face_encoding = face_recognition.face_encodings(dongkun_image)[0]

# Create arrays of known face encodings and their names
known_face_encodings = [
    obama_face_encoding,
    biden_face_encoding,
    xyy_face_encoding,
    gdg_face_encoding,
    dongkun_face_encoding,
]
known_face_names = [
    "Barack Obama",
    "Joe Biden",
    "Xiao Yueyue",
    "Guo Degang",
    "Dongkun",
]

print('Learned encoding for', len(known_face_encodings), 'images.')
```

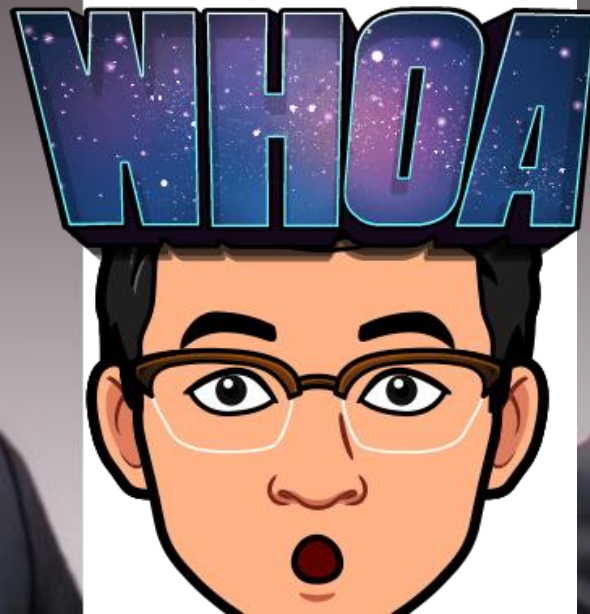


# Task5 -> Test on an image who not in the dataset

Add Dongkun's image in our dataset. Test again.



dongkun1.jpg



dongkun1.jpg

# Thank you!

Any questions?